

Makine Öğrenmesi

Ders 10

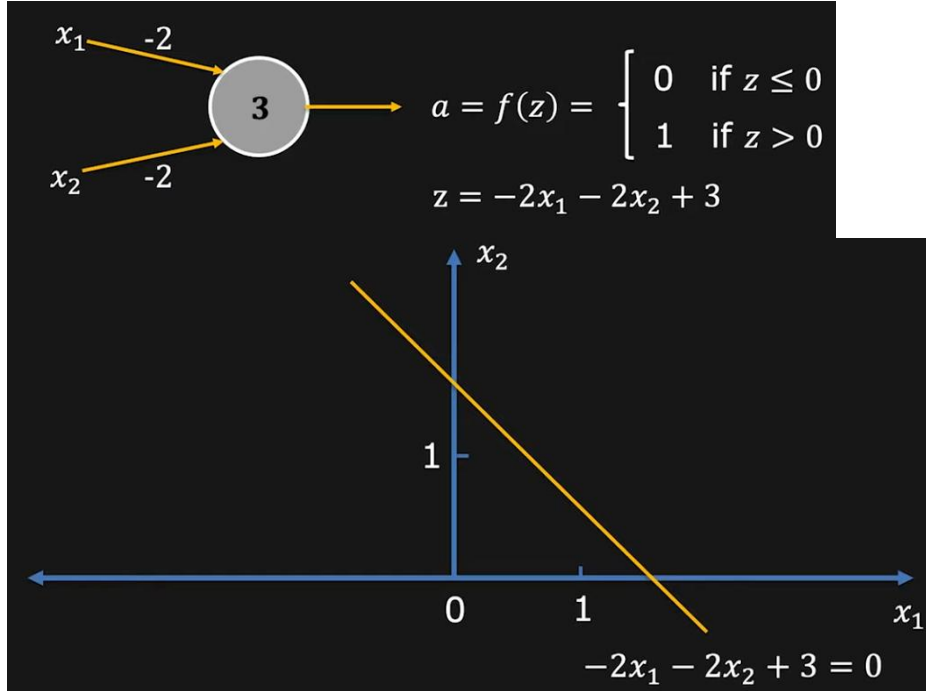


Bu Dersin İçeriği

- I. Çok Katmanlı Algılayıcı**
- II. Geriyayılım ile Eğitim**
- III. Çoklu Sınıflı Sınıflandırma**

I. Çok Katmanlı Algılayıcı

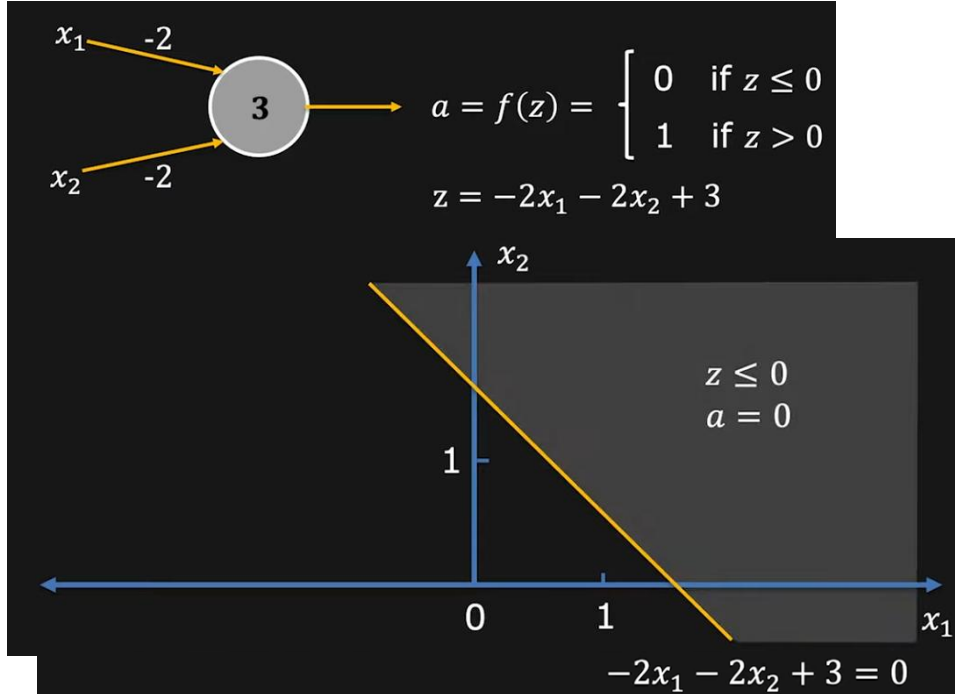
Bir sinir ağına daha yakından bakalım.



- Diyelim ki burası düz bir çizgi olsun.
- Burada olan şudur:Eğer x_1 , x_2 değerleri düz çizginin üzerinde veya sağında yer alıyorsa,bu durumda z , 0'dan küçük veya 0'a eşit olacaktır.
- Ve dolayısıyla, çıktı veya aktivasyon 0'a eşit olacaktır.
- Öte yandan, eğer çizginin sol tarafında yer alıyorsanız, z 0'dan büyük olacaktır.
- Ve çıktı veya aktivasyon 1'e eşit olacaktır.
- İşte algılayıcı (perceptron) bunu yapıyor.
- Doğrusal bir sınıflandırıcı gibi davranıyor.
- Doğrusal sınıflandırıcılar hakkında konuşmuştuk.

I. Çok Katmanlı Algılayıcı

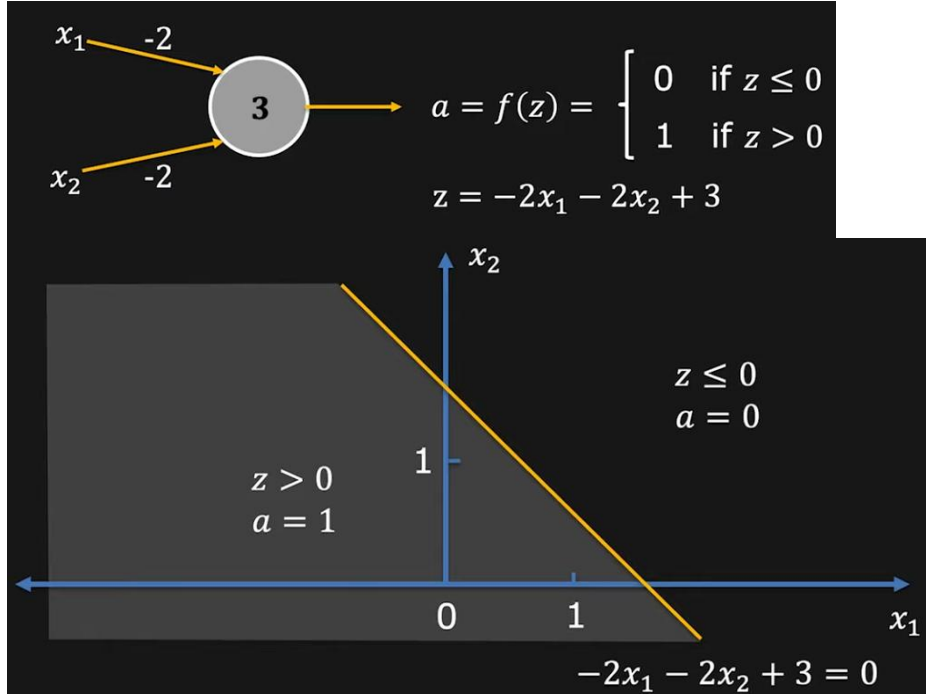
Bir sinir ağına daha yakından bakalım.



- Diyelim ki burası düz bir çizgi olsun.
- Burada olan şudur: Eğer x_1 , x_2 değerleri düz çizginin üzerinde veya sağında yer alıyorsa, bu durumda z , 0'dan küçük veya 0'a eşit olacaktır.
- Ve dolayısıyla, çıktı veya aktivasyon 0'a eşit olacaktır.
- Öte yandan, eğer çizginin sol tarafında yer alıyorsanız, z 0'dan büyük olacaktır.
- Ve çıktı veya aktivasyon 1'e eşit olacaktır.
- İşte algılayıcı (perceptron) bunu yapıyor.
- Doğrusal bir sınıflandırıcı gibi davranıyor.
- Doğrusal sınıflandırıcılar hakkında konuşmuştuk.

I. Çok Katmanlı Algılayıcı

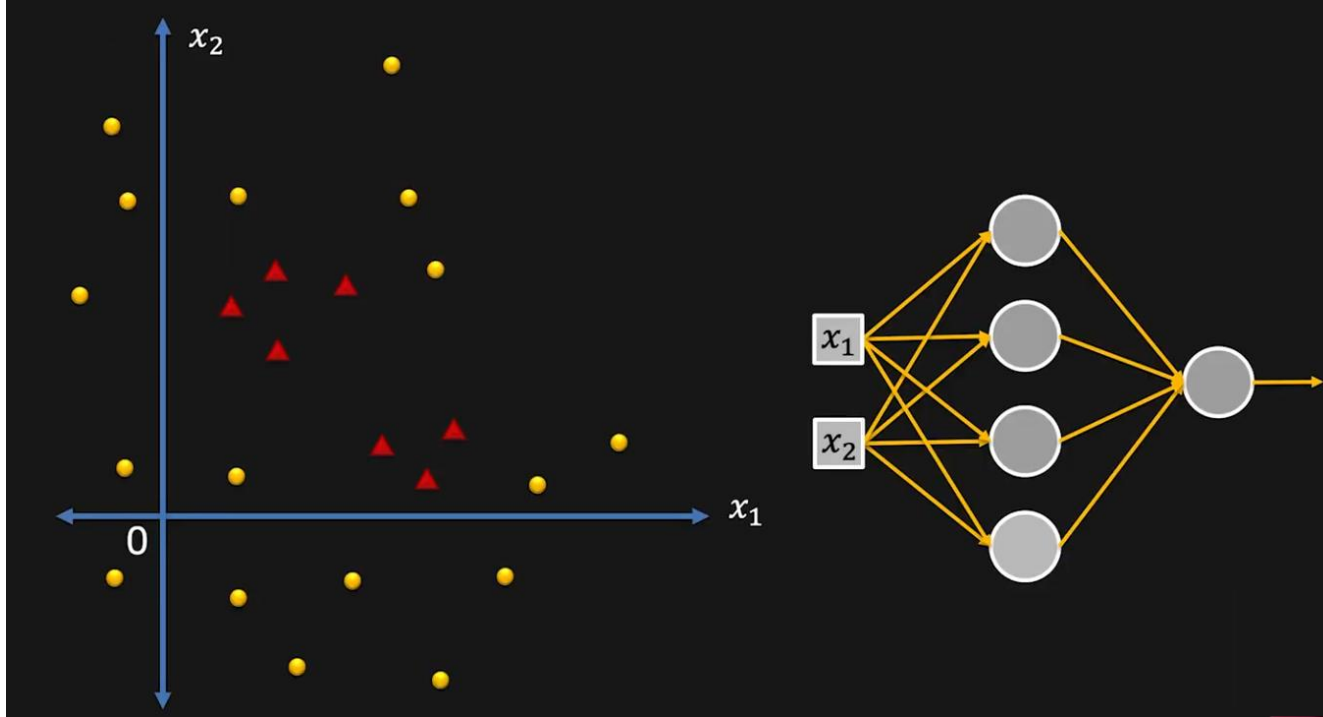
Bir sinir ağına daha yakından bakalım.



- Diyelim ki burası düz bir çizgi olsun.
- Burada olan şudur:Eğer x_1 , x_2 değerleri düz çizginin üzerinde veya sağında yer alıyorsa,bu durumda z , 0'dan küçük veya 0'a eşit olacaktır.
- Ve dolayısıyla, çıktı veya aktivasyon 0'a eşit olacaktır.
- Öte yandan, eğer çizginin sol tarafında yer alıyorsanız, z 0'dan büyük olacaktır.
- Ve çıktı veya aktivasyon 1'e eşit olacaktır.
- İşte algılayıcı (perceptron) bunu yapıyor.
- Doğrusal bir sınıflandırıcı gibi davranıyor.
- Doğrusal sınıflandırıcılar hakkında konuşmuştuk.

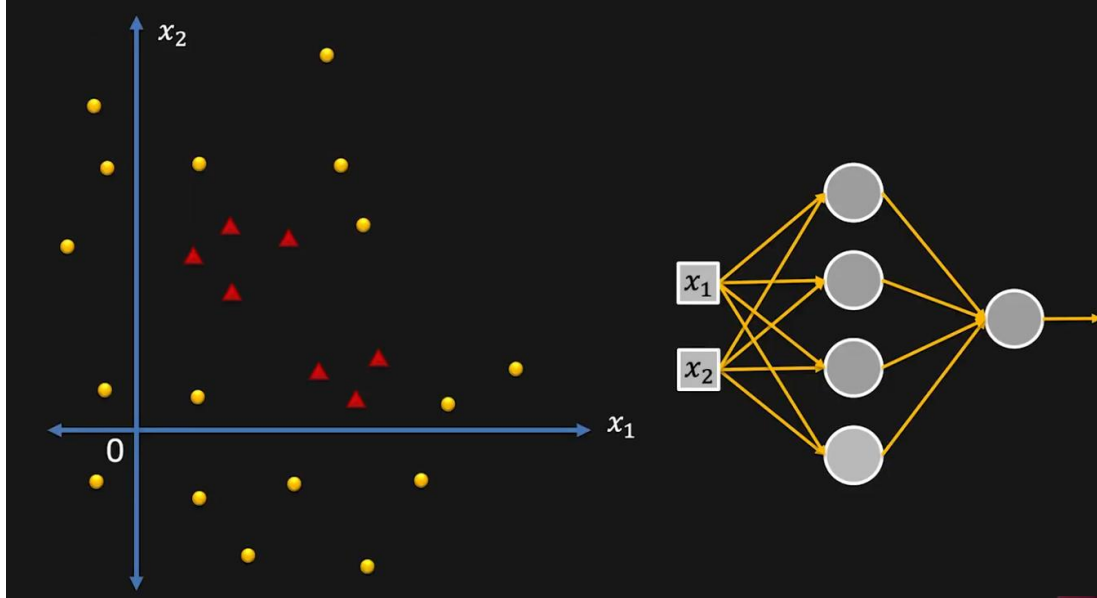
I. Çok Katmanlı Algılayıcı

Bir sinir ağına daha yakından bakalım.



I. Çok Katmanlı Algılayıcı

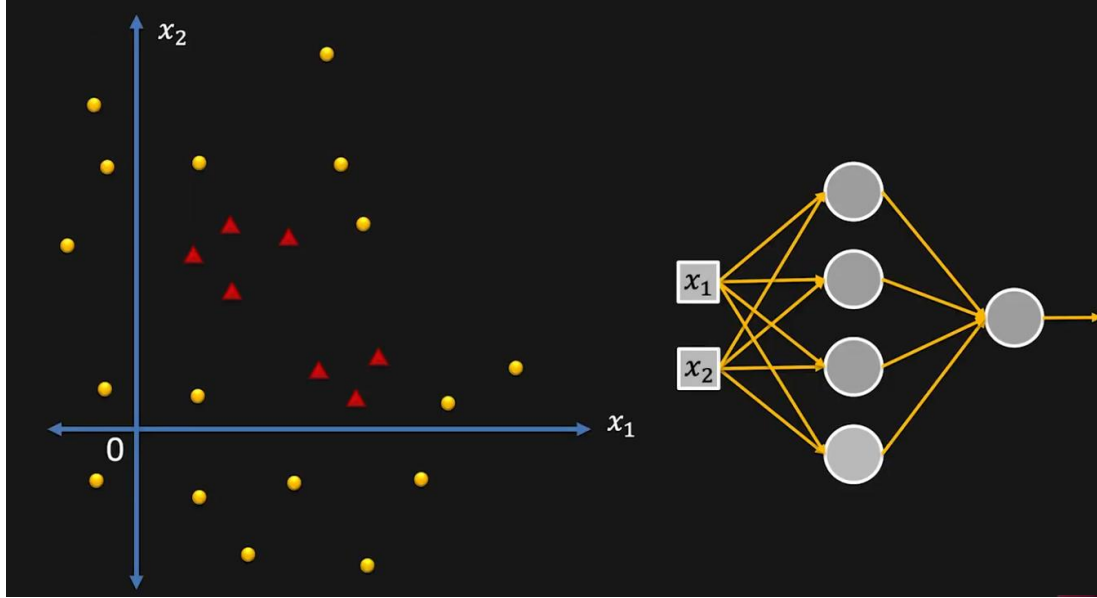
Bir sinir ağına daha yakından bakalım.



- Şimdi, yandaki probleme bakalım.
- Diyelim ki, yine x_1 , x_2 uzayınız var, ama şimdi burada kırmızı üçgenleriniz var ve sarı noktalarınız var.
- Bunları iki ayrı sınıf olarak ele alacağız.
- Ve esasen noktaları tahmin etmek istiyoruz yani bir noktanın kırmızı üçgenler sınıfına ait olup olmadığını belirlemek.

I. Sinir Ağları

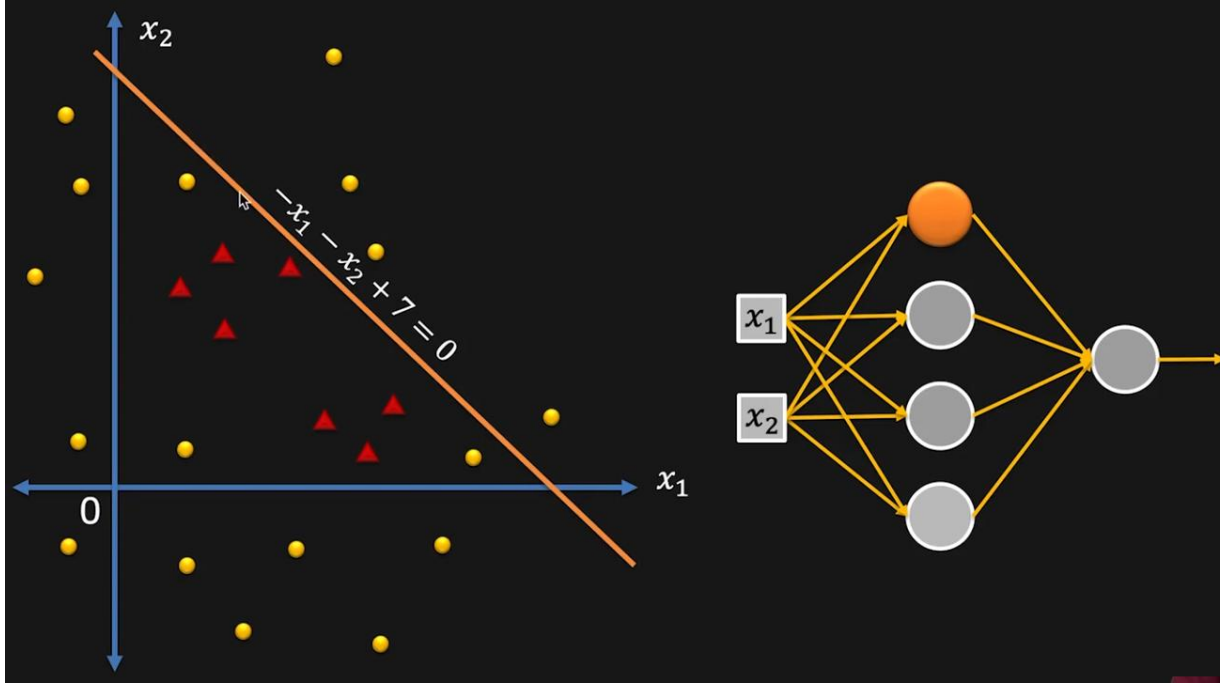
Bir sinir ağına daha yakından bakalım.



- Şimdi bir bölgemiz var ve kırmızı üçgenlerin etrafındaki bu bölgeyi bir şekilde tanımlayabilecek doğrusal bir sınıflandırıcı veya bir dizi doğrusal sınıflandırıcı oluşturmak istiyoruz.

I. Çok Katmanlı Algılayıcı

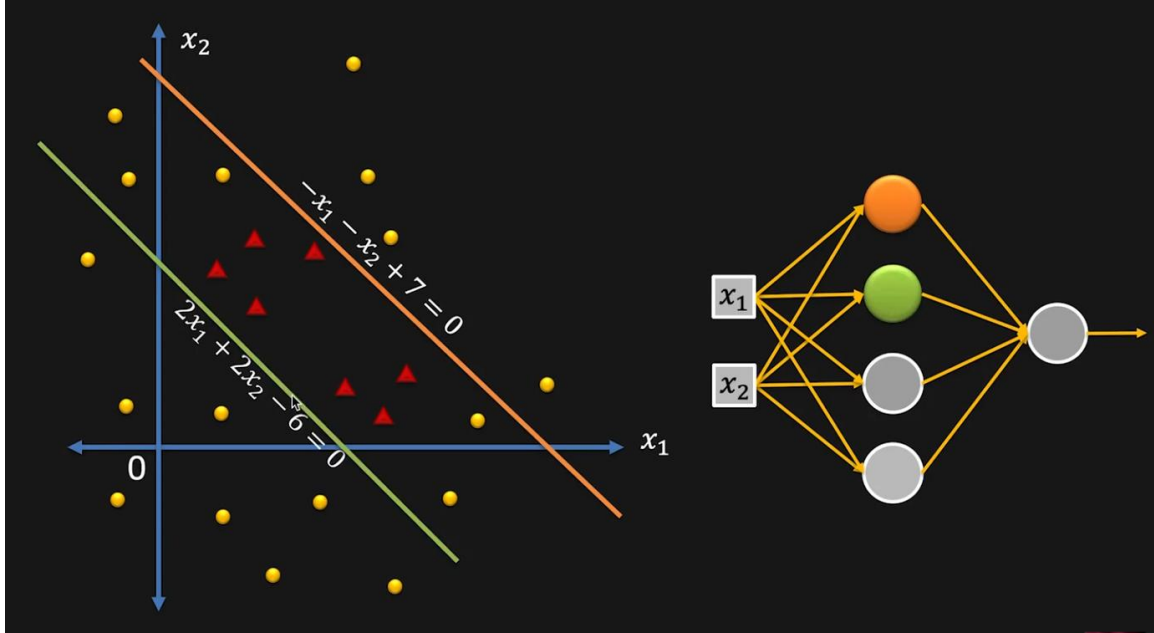
Bir sinir ağına daha yakından bakalım.



Turuncu perceptronu
doğrusunu çizelim.

I. Çok Katmanlı Algılayıcı

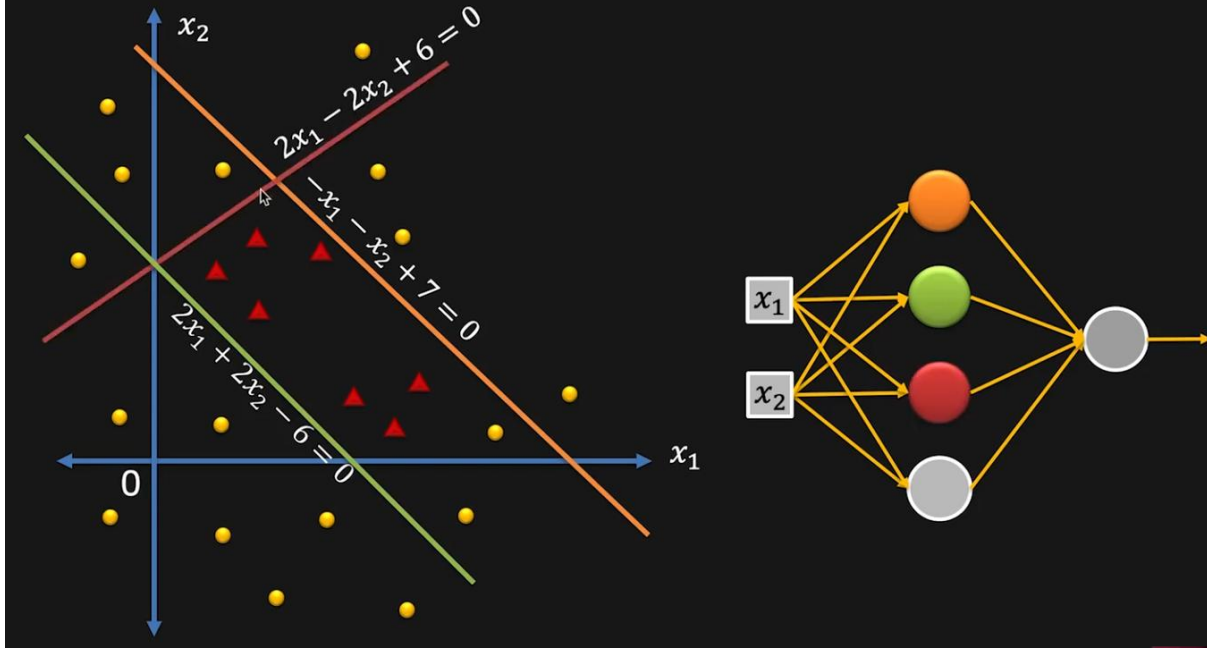
Bir sinir ağına daha yakından bakalım.



- Yeşil perceptronu doğrusunu çizelim.

I. Çok Katmanlı Algılayıcı

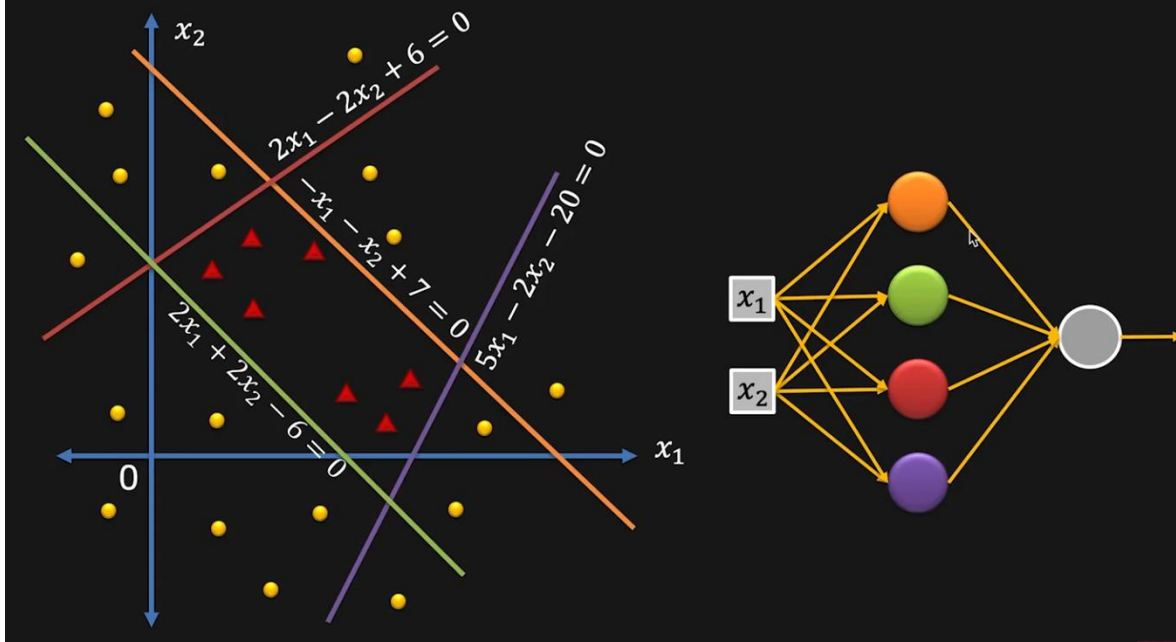
Bir sinir ağına daha yakından bakalım.



- Kırmızı perceptronu doğrusunu çizelim.

I. Çok Katmanlı Algılayıcı

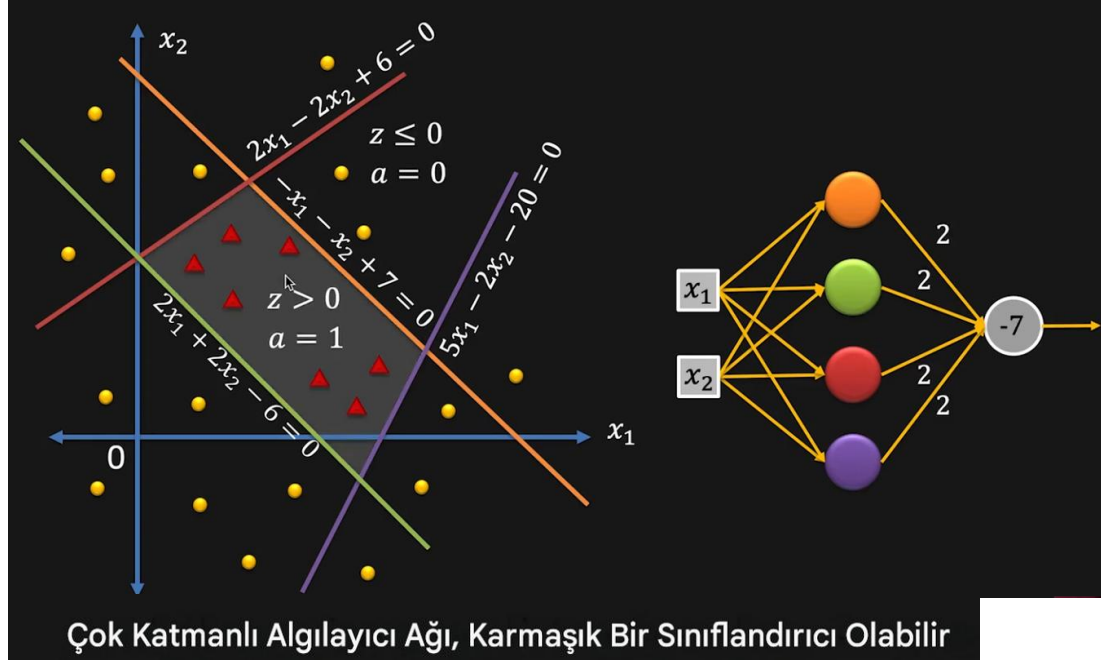
Bir sinir ağına daha yakından bakalım.



- Mor perceptronu doğrusunu çizelim.

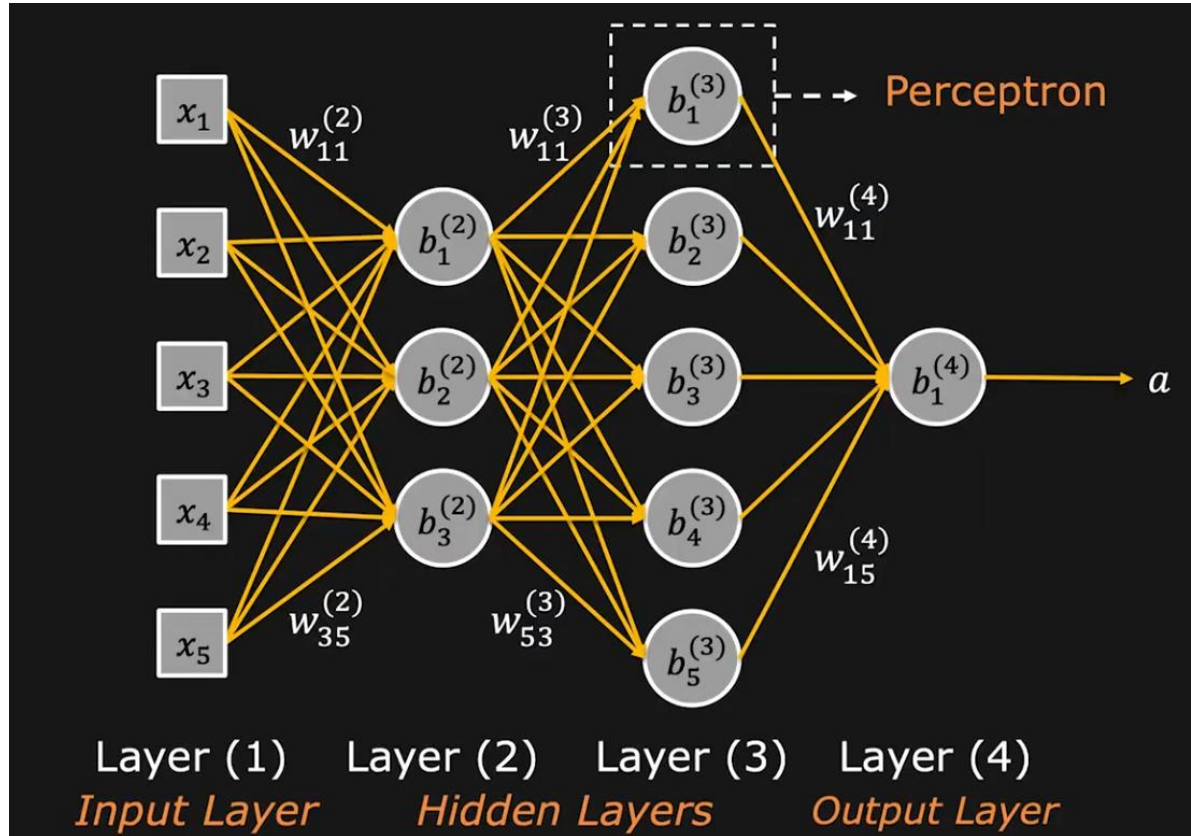
I. Çok Katmanlı Algılayıcı

Bir sinir ağına daha yakından bakalım.



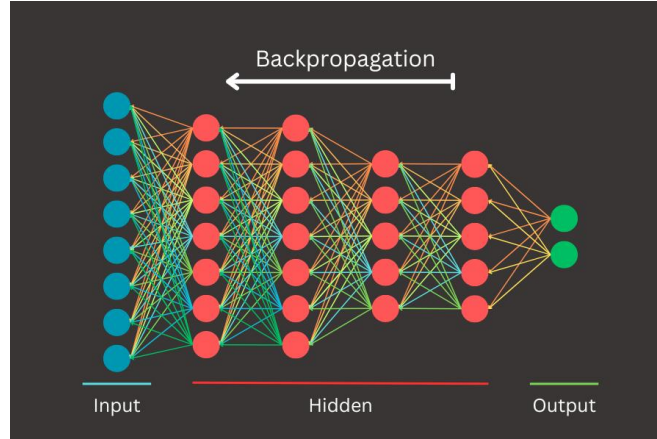
- Eğer tüm algılayıcılar 1 üretiyorsa, burada bir çıkış algılayıcısı oluşturabilirsiniz;
- Bu algılayıcı yalnızca tüm girdiler 1 olduğunda 1 çıktısı üretir.
- Ve bu, örneğin bu ağırlıkları ve bu sapmayı kullanarak yapılır.
- Aynı şeyi yapmak için başka ağırlıklar ve sapmalar da kullanabilirsiniz.
- Bu şekilde daha karmaşık bir bölge için doğrusal bir sınıflandırıcı oluşturabilirsiniz.
- Bunu iki boyutlu uzayda gösterdik, ancak bu herhangi bir boyutta olabilir.

I. Çok Boyutlu Algılayıcı



II. Geriyayılım ile Eğitim

1. Geriye yayılım, sinir ağları için en yaygın eğitim algoritmasıdır.
2. Geri yayılım, bu hatayı ağdaki her bir ağırlığa (parametreye) nasıl dağıtacağımızı ve bu ağırlıkların hatayı azaltacak yönde nasıl güncelleyeceğimizi hesaplayan bir yöntemdir.
3. Temelde, kalkülüsteki zincir kuralını (chain rule) yinelemeli olarak uygulayarak çalışır.
4. Çok katmanlı sinir ağları için gradyan inişini (gradient descent) uygulanabilir hale getirir.



II. Geriyayılım ile Eğitim

1. Geriye yayılım eğitim algoritması, kaybı en aza indirmek (teorik olarak yok etmek) amacıyla model ağırlıklarını ayarlamak için türevini (gradyan-gradient) hesaplama kavramını kullanır.
2. Geri yayılma, sinir ağlarında ağırlıkları güncelleyebilmek için hata (kayıp) fonksiyonunun türevini (gradyanını) hesaplayan algoritmadır.
3. Birçok makine öğrenimi kod kütüphanesi (Keras gibi) geri yayılımı otomatik olarak işler, böylece altta yatan hesaplamaların hiçbirini kendimiz yapmamız gerekmez.

II. Geriyayılım ile Eğitim

Adımlar:

1. İleri yayılım (forward pass): Giriş verisi ağdan geçirilir, çıktı (tahmin) hesaplanır.
2. Kayıp (loss) hesaplama: Gerçek değer ile tahmin arasındaki fark belirlenir (örneğin MSE, cross-entropy).
3. Geri yayılım (backward pass): Hata, zincir kuralı (chain rule) ile katman katman geriye doğru türevlenerek dağıtılır. Her ağırlık için $\partial \text{Loss} / \partial \text{Weight}$ hesaplanır.
4. Ağırlık güncellemesi: Genellikle Stokastik Gradyan İnişi (SGD) ile yapılır:

$$W \leftarrow W - \eta \cdot \frac{\partial L}{\partial W}$$

Burada η öğrenme oranıdır.

II. Geriyayılım ile Eğitim

1. İleri yayılım (forward Propagation-Pass):

1. Giriş verisi (X) ağın ilk katmanına verilir.
2. Her bir katmanda, bir önceki katmanın çıktıları ($a^{(l-1)}$) alınır, o katmanın ağırlıkları ($W^{(l)}$) ile çarpılır, bias terimi ($b^{(l)}$) eklenir ($z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$) ve sonuç bir aktivasyon fonksiyonundan (σ) geçirilir ($a^{(l)} = \sigma(z^{(l)})$).
3. Bu işlem, ağın çıktı katmanına ($a^{(L)}$) ulaşana kadar devam eder.
4. Son olarak, ağın çıktısı ($a^{(L)}$) ile gerçek hedef değerler (y) kullanılarak bir kayıp fonksiyonu ($J(W, b)$ veya L) hesaplanır. Örneğin, kareler toplamı hatası veya çapraz entropi (cross-entropy) gibi.

II. Geri yayılım ile Eğitim

1. Geri Yayılım (Backward Propagation):

1. Amaç: **Kayıp fonksiyonunun (J) ağıdaki her bir ağırlık ($W_{ij}^{(l)}$) ve bias'a ($b_i^{(l)}$) göre kısmi türevini ($\frac{\partial J}{\partial W_{ij}^{(l)}}$ ve $\frac{\partial J}{\partial b_i^{(l)}}$) hesaplamak. Bu türevler, "gradyanlar" olarak adlandırılır ve her bir parametrenin kaybı ne kadar etkilediğini gösterir.**
2. Çıktı Katmanı Gradyanı: **İlk olarak, kayıp fonksiyonunun çıktı katmanındaki aktivasyonlara ($a^{(L)}$) ve ardından pre-aktivasyonlara ($z^{(L)}$) göre türevi hesaplanır. Bu, "hata sinyali" $\delta^{(L)}$ olarak ifade edilebilir: $\delta^{(L)} = \frac{\partial J}{\partial a^{(L)}} \odot \sigma'(z^{(L)})$ Burada $\odot$ eleman bazında çarpımı, $\sigma'(z^{(L)})$ ise çıktı katmanındaki aktivasyon fonksiyonunun türevini ifade eder.**

II. Geri yayılım ile Eğitim

1. Geri Yayılım (Backward Propagation):

3. Gradyanları Geriye Doğru Yayma: **Zincir kuralı kullanılarak, bu hata sinyali bir önceki katmana (L-1) doğru yayılır:** $\delta^{(l)} = ((W^{(l+1)})^T \delta^{(l+1)}) \odot \sigma'(z^{(l)})$ **Bu işlem, ağın ilk gizli katmanına ulaşana kadar tekrarlanır. $((W^{(l+1)})^T \delta^{(l+1)})$ terimi, bir sonraki katmanın ağırlıklı hatasının mevcut katmana nasıl yansıdığını gösterir.**
4. Ağırlık ve Bias Gradyanlarının Hesaplanması: **Her katman için hata sinyali $\delta^{(l)}$ hesaplandıktan sonra, o katmandaki ağırlıklar ve biaslar için gradyanlar şu şekilde bulunur:** $\frac{\partial J}{\partial W^{(l)}} = \delta^{(l)} (a^{(l-1)})^T \frac{\partial J}{\partial b^{(l)}} = \delta^{(l)}$ **(genellikle $\delta^{(l)}$ 'nin elemanlarının toplanması veya ortalaması alınır, implementasyona göre değişir)**

II. Geriyayılım ile Eğitim

1. Geri Yayılım (Backward Propagation):

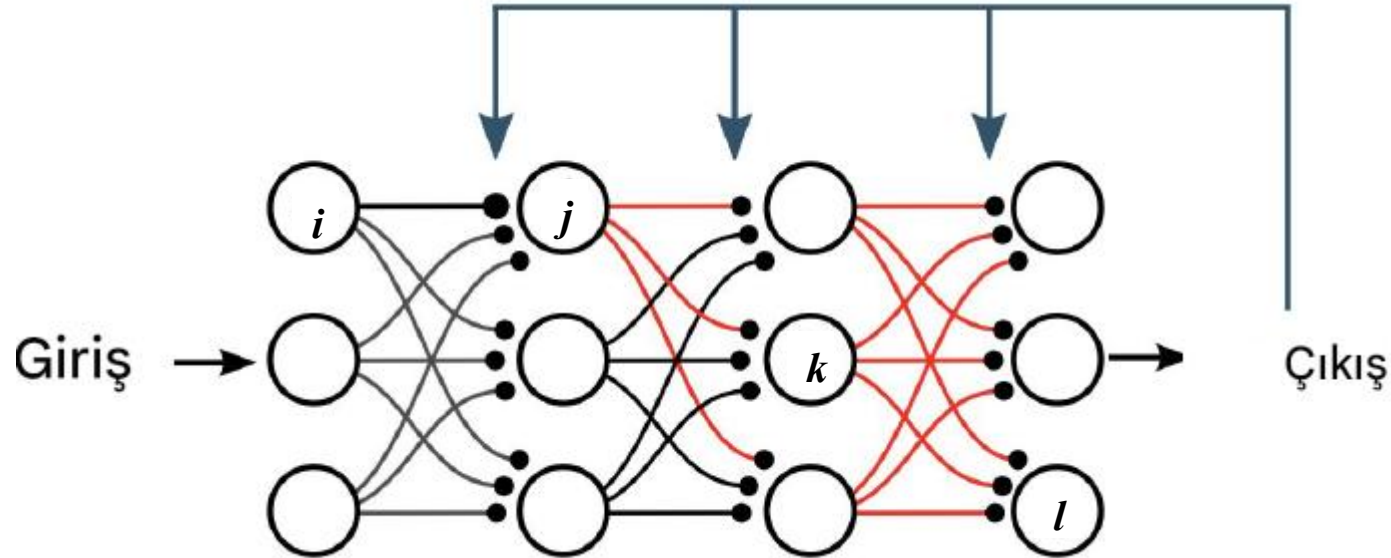
5. Ağırlık Güncellemesi: **Gradyanlar hesaplandıktan sonra, ağırlıklar ve biaslar bir optimizasyon algoritması (genellikle Gradyan İniş - Gradient Descent veya varyantları) kullanılarak güncellenir:** $W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial J}{\partial W^{(l)}}$ $b^{(l)} \leftarrow b^{(l)} - \eta \frac{\partial J}{\partial b^{(l)}}$
Burada η öğrenme oranıdır (learning rate).

2. Bu sayede Geri yayılım, çıktıdaki hatanın sorumluluğunu ağ boyunca geriye doğru, her bir ağırlığa katkısı oranında dağıtır.

II. Geriyayılım ile Eğitim

Vektör gradient hesaplanır ve ağırlıklar yeniden dağıtılır.

$$\Delta W = \eta \cdot \frac{\partial L}{\partial W}$$



II. Geriyayılım ile Eğitim - Sorunlar

1. Kaybolan Gradyanlar: Alt sinir ağı katmanlarının (giriş katmanına daha yakın olanlar) gradyanları çok küçük olabilir.
2. Derin ağlarda (birden fazla gizli katmanı olan ağlar), bu gradyanları hesaplamak birçok küçük terimin çarpımını almayı gerektirebilir.
3. Gradyan değerleri alt katmanlar için 0'a yaklaştığında, gradyanların “yok olduğu” söylenir.
4. Kaybolan gradyanlara sahip katmanlar çok yavaş eğitilir veya hiç eğitilmez. Bu katmanlar karmaşık özellikleri öğrenemez, bu da modelin performansını düşürür.
5. Özellikle sigmoid, tanh gibi aktivasyon fonksiyonlarında türevleri 0–1 arasında olduğundan, çarpıldıkça küçülür.
6. Çözüm:
 - a) ReLU aktivasyon fonksiyonu kaybolan gradyanları önlemeye yardımcı olabilir.
 - b) Batch normalization, residual connections (skip connections) (özellikle ResNet’lerde),
 - c) İyi ağırlık başlangıç yöntemleri (Xavier, Kaiming He initialization)

II. Geri yayılım ile Eğitim - Sorunlar

1. Patlayan Gradyanlar: Bir ağdaki ağırlıklar çok büyükse, alt katmanlar için gradyanlar birçok büyük terimin çarpımını içerir.
2. Bu durum yakınsamak için çok büyük olan gradyanlar anlamına gelen patlayan gradyanlara neden olaabilir.
3. Gradyanlar geri yayılımda aşırı büyür, sayı kararsızlığına yol açar.
4. Ağırlıklar fırlar, model bozulur (NaN oluşabilir).
5. Çözüm:
 - a) Gradyan kırpma (gradient clipping): Gradyanların normu (büyüklüğü) belirli bir eşik değeri aşarsa, gradyanlar bu eşik değerine ölçeklenir.
 - b) Toplu iş boyutu normalleştirme (Batch normalization), öğrenme hızını düşürmek gibi patlayan gradyanları önlemeye yardımcı olabilir.
 - c) Ağırlık regularizasyonu (L2) ve normalizasyon teknikleri,
 - d) Uygun aktivasyon ve başlangıç yöntemleri.

II. Geri yayılım ile Eğitim - Sorunlar

1. Ölü ReLU Birimleri: ReLU ($\max(0,x)$) aktivasyon fonksiyonunu kullanan bir nöron, ağırlıklı girdisi (z) sürekli olarak negatif olduğunda "ölü" hale gelebilir.
2. Bir ReLU birimi için ağırlıklı toplam 0'ın altına düştüğünde, ReLU birimi sıkışabilir ve 0 çıktısı vererek ağıın çıktısına hiçbir katkıda bulunmaz.
3. ReLU'nun türevi $x < 0$ için 0 olduğundan, bu nörondan geriye doğru akan gradyan da 0 olur.
4. Ölü ReLU birimi artık hiçbir veriye tepki vermez ve öğrenme sürecine katkıda bulunmaz.
5. Gradyanlar artık geri yayılım sırasında içinden akamaz.
6. Bir gradyan kaynağı kesildiğinde, ReLU'ya giriş, ağırlıklı toplamı tekrar 0'ın üzerine çıkaracak kadar asla değişmeyebilir.
7. Ağıın kapasitesi etkili bir şekilde azalır.

II. Geriyayılım ile Eğitim - Sorunlar

1. Ölü ReLU Birimleri:

2. Çözüm:

- a) **Leaky ReLU:** $x < 0$ için $f(x) = \alpha x$ (örneğin $\alpha = 0.01$) gibi küçük, sıfır olmayan bir eğim kullanır. Bu sayede negatif girdiler için de sıfır olmayan bir gradyan akar.
- b) **Exponential Linear Unit (ELU):** Negatif değerler için yumuşak bir doygunluğa sahiptir ve sıfır olmayan gradyanlar sağlar.
- c) **Dikkatli Ağırlık Başlatma ve Düşük Öğrenme Oranları:** ReLU birimlerinin başlangıçta veya eğitim sırasında ölmesini engellemeye yardımcı olabilir.
- d) **Adam Optimizer gibi Adaptif Öğrenme Oranı Algoritmaları:** Öğrenme oranını dinamik olarak ayarlayarak bu sorunu azaltabilir.

II. Geriyayılım ile Eğitim - Sorunlar

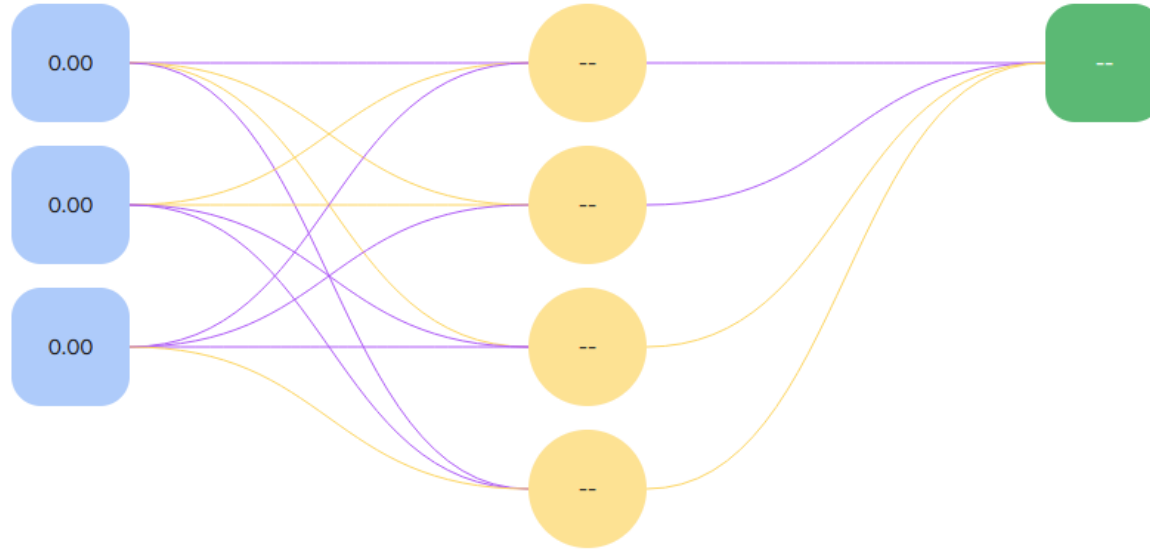
1. Bırakma Düzenlemesi (Dropout regularization) adı verilen bir başka regularization biçimi de sinir ağları için kullanışlıdır.
2. Eğitim sırasında her iterasyonda rastgele bazı nöronlar geçici olarak devre dışı bırakılır.
3. Örnek: Her nöronun %50 olasılıkla devre dışı kalması.
4. Tek bir gradyan adımı için bir ağıdaki birim aktivasyonlarını rastgele bırakarak çalışır.
5. Bu düzenleme ağı fazla bağımlılıkları kırmak, nöronların genelleştirilebilir temsiller öğrenmesini sağlamak ve aşırı uyumu azaltmak için yapılabilir.
6. Ne kadar çok bırakırsak, düzenleme o kadar güçlü olur:
 - a) 0.0 = Bırakma düzenlemesi yok.
 - b) 1.0 = Tüm düğümler atılır. Model hiçbir şey öğrenmez.
 - c) 0,0 ve 1,0 arasındaki değerler = Daha kullanışlıdır.

II. Geriyayılım ile Eğitim - Sorunlar

İşlem	Ne Olduğu?	Sonuç	Çözüm
Kaybolan Gradyan	Gradyanlar 0'a gider.	Öğrenme durur	ReLU, batch norm, skip connection
Patlayan Gradyan	Gradyanlar ∞ 'a gider.	Öğrenme kararsız	Gradyan kırpma, regularization
Ölü ReLU	Nöron hep 0 üretir	Öğrenmeye katkı sıfır	Leaky ReLU, daha iyi initialization
Dropout	Rastgele nöron silme	Aşırı uyum azalır	% oran dikkatli seçilmeli

II. Geri yayılım ile Eđitim Örnek Uygulama

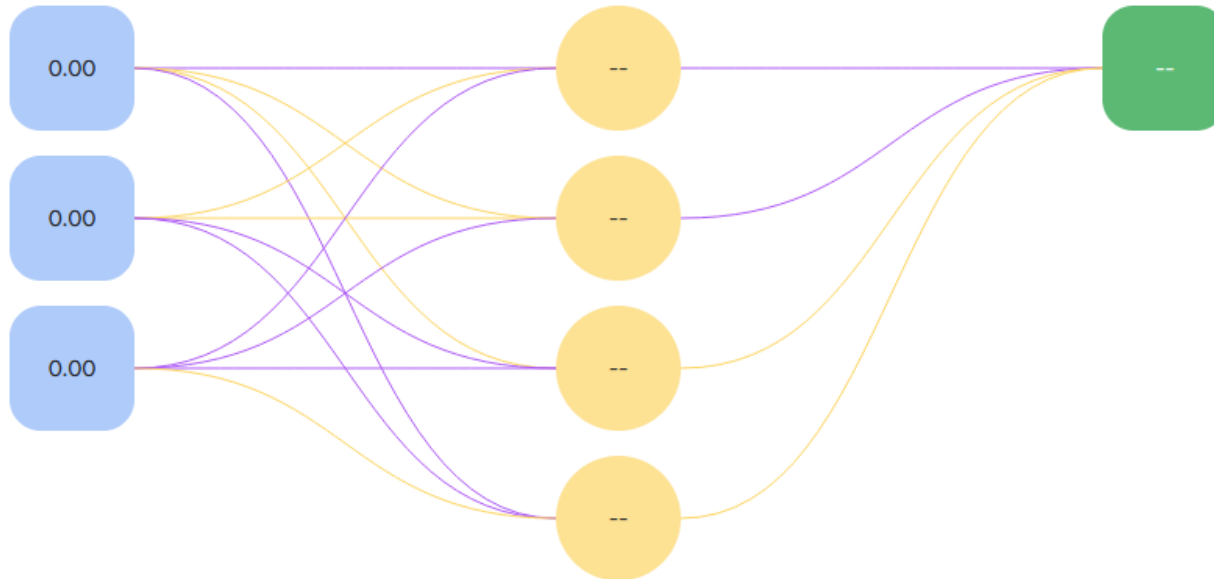
1. ReLU aktivasyon fonksiyonu olan 3 giriřili bir gizli katmanı ve gizli katmanda 4 nöronu olan bir sinir ađımız olsun.
2. Sinir ađı modeline üç giriř özelliđi için deđerlerin tümü 0,00'dır.



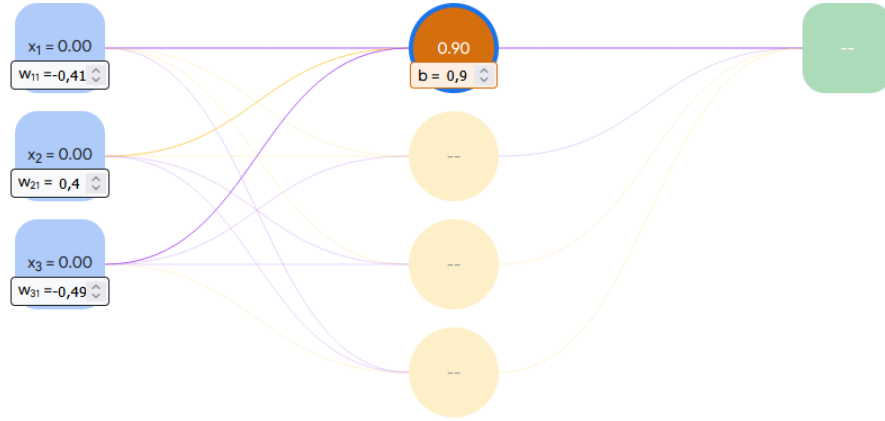
II. Geri yayılım ile Eđitim rnek Uygulama

1. Ne tr bir ıktı deęeri retileceęini dřnelim: pozitif mi negatif mi veya 0 mı?

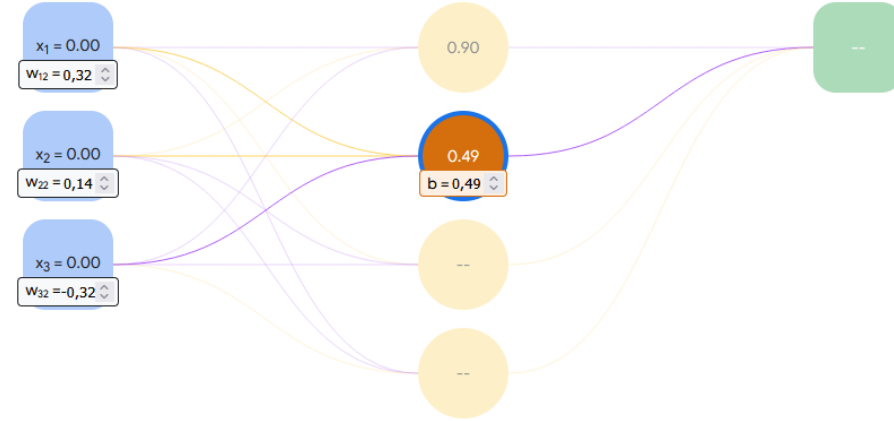
- a) Pozitif ıkıř deęeri
- b) Negatif ıkıř deęeri
- c) 0 ıkıř deęeri



II. Geri yayılım ile Eğitim Örnek Uygulama

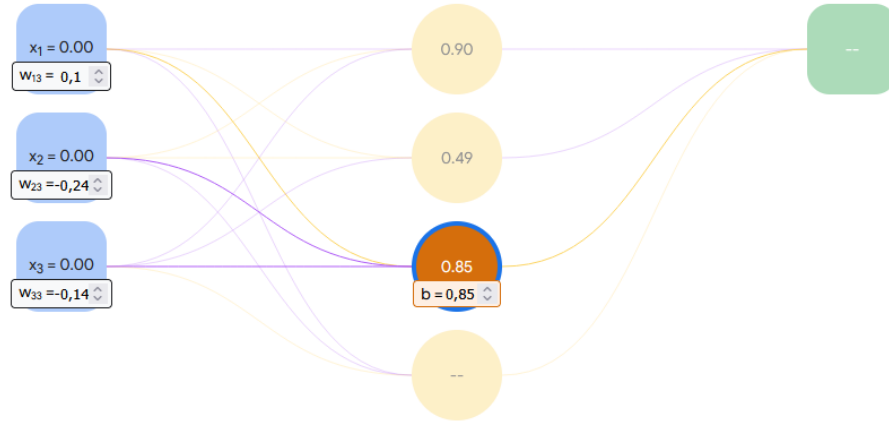


$$\begin{aligned} &= \text{ReLU}(w_{11} x_1 + w_{21} x_2 + w_{31} x_3 + b) \\ &= \text{ReLU}(-0.41 * 0.00 + 0.40 * 0.00 + -0.49 * 0.00 + 0.90) \\ &= \text{ReLU}(0.00 + 0.00 + 0.00 + 0.90) \\ &= \text{ReLU}(0.90) \\ &= 0.90 \end{aligned}$$

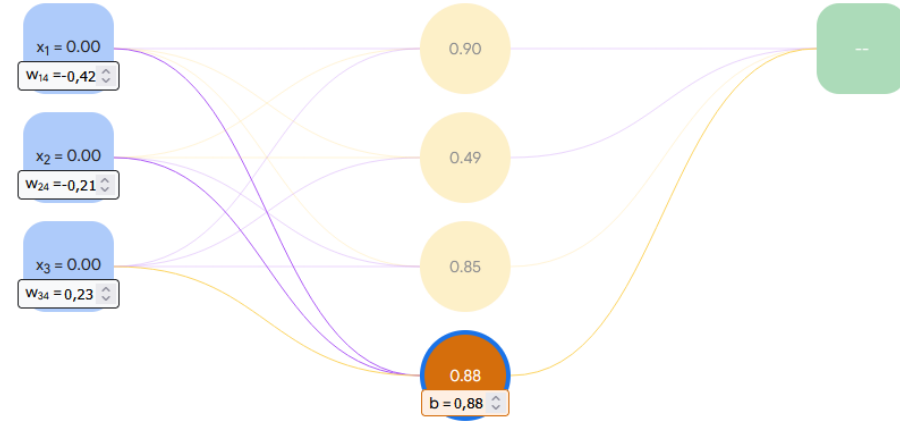


$$\begin{aligned} &= \text{ReLU}(w_{12} x_1 + w_{22} x_2 + w_{32} x_3 + b) \\ &= \text{ReLU}(0.32 * 0.00 + 0.14 * 0.00 + -0.32 * 0.00 + 0.49) \\ &= \text{ReLU}(0.00 + 0.00 + 0.00 + 0.49) \\ &= \text{ReLU}(0.49) \\ &= 0.49 \end{aligned}$$

II. Geri yayılım ile Eğitim Örnek Uygulama

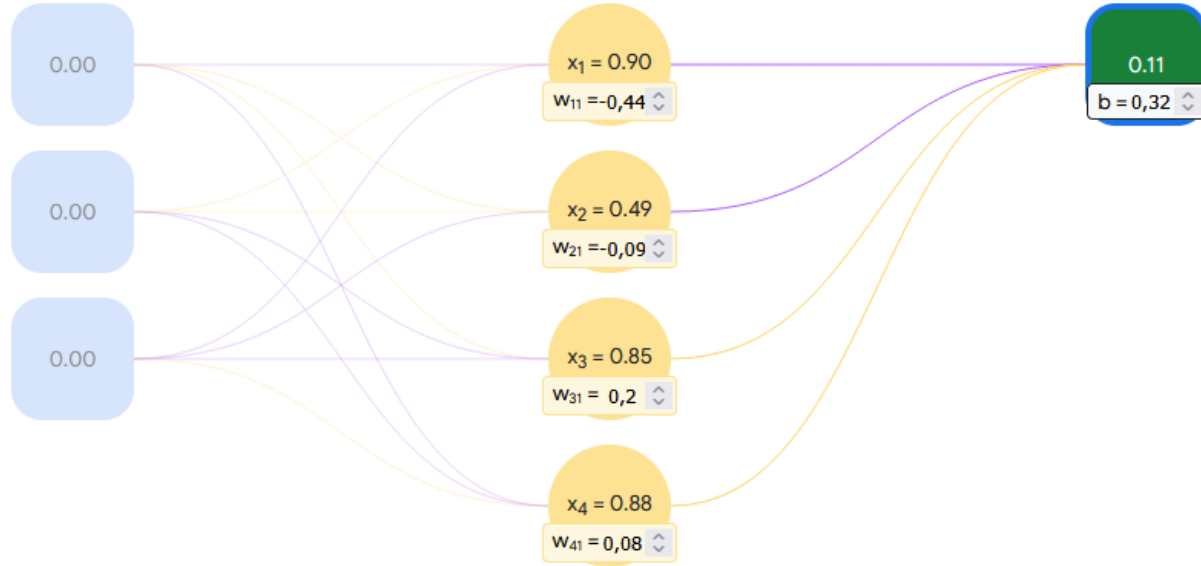


$$\begin{aligned} &= \text{ReLU}(w_{13} x_1 + w_{23} x_2 + w_{33} x_3 + b) \\ &= \text{ReLU}(0.10 * 0.00 + -0.24 * 0.00 + -0.14 * 0.00 + 0.85) \\ &= \text{ReLU}(0.00 + 0.00 + 0.00 + 0.85) \\ &= \text{ReLU}(0.85) \\ &= 0.85 \end{aligned}$$



$$\begin{aligned} &= \text{ReLU}(w_{14} x_1 + w_{24} x_2 + w_{34} x_3 + b) \\ &= \text{ReLU}(-0.42 * 0.00 + -0.21 * 0.00 + 0.23 * 0.00 + 0.88) \\ &= \text{ReLU}(0.00 + 0.00 + 0.00 + 0.88) \\ &= \text{ReLU}(0.88) \\ &= 0.88 \end{aligned}$$

II. Geri yayılım ile Eğitim Örnek Uygulama



$$\begin{aligned} &= \text{ReLU}(w_{11} x_1 + w_{21} x_2 + w_{31} x_3 + w_{41} x_4 + b) \\ &= \text{ReLU}(-0.44 * 0.90 + -0.09 * 0.49 + 0.20 * 0.85 + 0.08 * 0.88 + 0.32) \\ &= \text{ReLU}(-0.40 + -0.05 + 0.17 + 0.07 + 0.32) \\ &= \text{ReLU}(0.11) \\ &= 0.11 \end{aligned}$$

II. Geri yayılım ile Eğitim Örnek Uygulama

1. Elde edeceğiniz tam çıktı değeri, ağırlık ve sapma parametrelerinin rastgele nasıl başlatıldığına bağlı olarak değişecektir.
2. Ancak, giriş katmanındaki her nöron 0 değerine sahip olduğundan, gizli katman düğüm değerlerini hesaplamak için kullanılan ağırlıkların tümü sıfırlanacaktır.
3. Örneğin, ilk gizli katman düğümü hesaplaması şöyle olacaktır:

$$y = \text{ReLU}(w_{11} * 0.00 + w_{21} * 0.00 + w_{31} * 0.00 + b), \quad y = \text{ReLU}(b)$$

4. Dolayısıyla, her bir gizli katman düğümünün değeri, b negatifse 0, b 0 veya pozitifse b'nin kendisi olacak olan önyargının (b) ReLU değerine eşit olacaktır.
5. Çıkış düğümünün değeri daha sonra aşağıdaki gibi hesaplanacaktır:

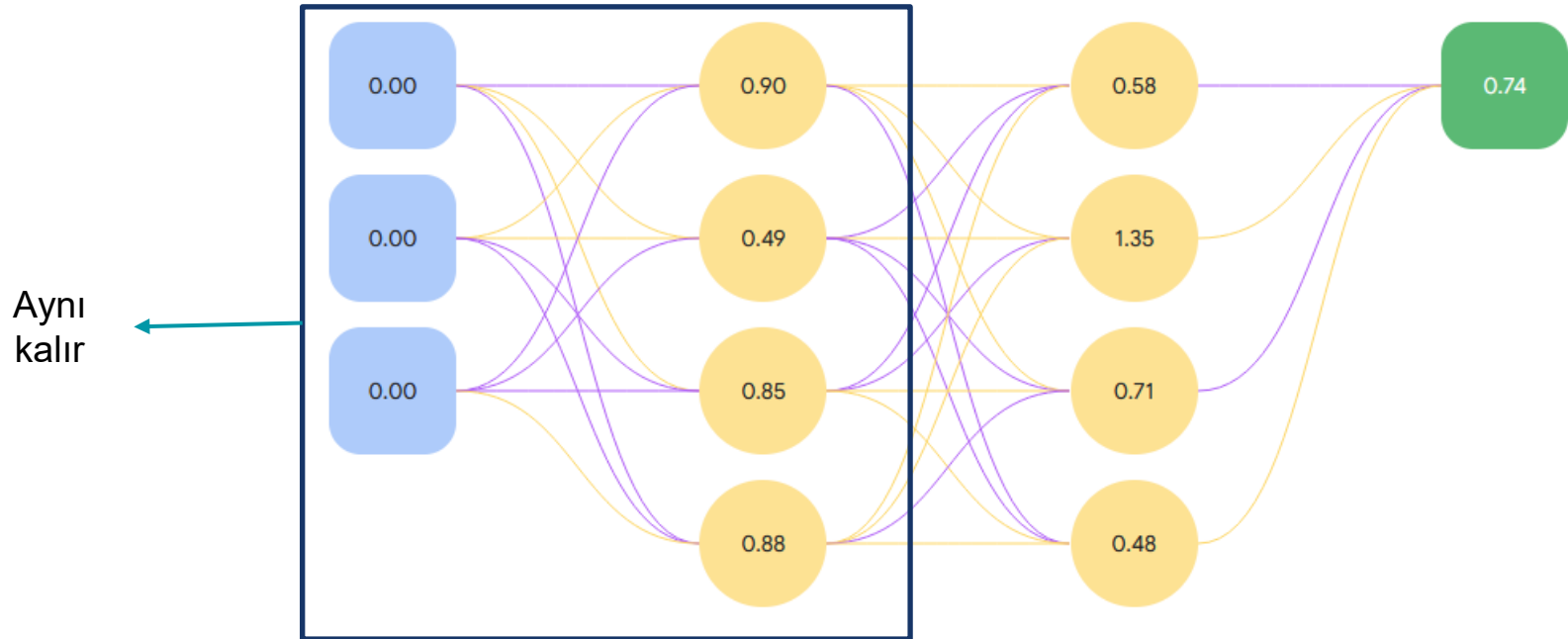
$$y = \text{ReLU}(w_{11} * x_{11} + w_{21} * x_{21} + w_{31} * x_{31} + w_{41} * x_{41} + b)$$

II. Geriyayılım ile Eğitim Örnek Uygulama

1. Sinir ağına ilk gizli katmandan sonra bir gizli katman daha ekleyip ve bu yeni katmana 3 düğüm verelim. Tüm giriş ve ağırlık/önyargı parametrelerini aynı tutarsak, diğer hangi düğümlerin hesaplamaları etkilenir?
 - a) Giriş düğümleri hariç ağdaki tüm düğümler
 - b) Sadece ilk gizli katmandaki düğümler
 - c) Sadece çıkış düğümü

II. Geri yayılım ile Eđitim Örnek Uygulama

1. Sadece çıkış düğümü deđiřir. Bu sinir ađı için çıkarım “ileri beslemeli” olduđundan (hesaplamalar bařtan sona ilerler), ađa yeni bir katman eklenmesi yalnızca yeni katmandan sonraki düğümleri etkileyecektir, ondan öncekileri deđil.

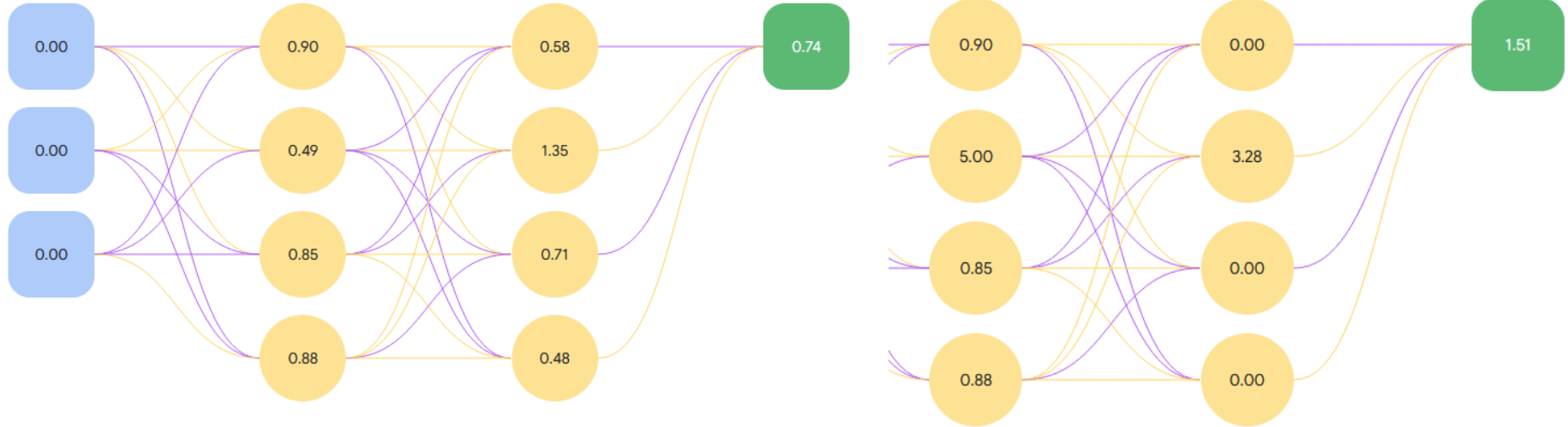


II. Geriyayılım ile Eğitim Örnek Uygulama

1. Ağ yapısında ilk gizli katmanındaki ikinci düğümüne ait olan w_{12} ağırlığının değerini değiştirirsek bazı girdi değerleri için diğer hangi düğümlerin hesaplamaları etkilenebilir?
 - a) Hiçbiri
 - b) Birinci gizli katmandaki ikinci düğüm, ikinci gizli katmandaki tüm düğümler ve çıkış düğümü.
 - c) Birinci gizli katmandaki, ikinci gizli katmandaki ve çıktı katmanındaki tüm düğümler.

II. Geri yayılım ile Eğitim Örnek Uygulama

1. w_{12} ağırlığı için değerini 5,00 olarak değiştirelim ve gerekli hesaplamaları tekrardan yapalım.



2. Birinci gizli katmandaki diğer düğümler için değer hesaplamaları parametre olarak w_{12} içermez, bu nedenle etkilenmezler.
3. Değişen düşümler şu şekilde olacaktır;
4. Birinci gizli katmandaki ikinci düğüm, ikinci gizli katmandaki tüm düğümler ve çıkış düğümü.

II. Geriyayılım ile Eğitim Örnek Uygulama

1. Kategorik veriler konusundaki Özellik çaprazlama alıştırmalarında, doğrusal olmayan verilere uyması için özellik çaprazlamalarını manuel olarak oluşturmuştuk.
2. Şimdi, eğitim sırasında doğrusal olmayan verilere nasıl uyum sağlayacağını otomatik olarak öğrenebilen bir sinir ağı oluşturup oluşturamayacağınızı görecelim.
3. Playground'da turuncu noktaları mavi noktalardan ayırabilen ve hem eğitim hem de test verilerinde 0,2'den daha az bir kayıp elde eden bir sinir ağı yapılandırabilir miyiz?
4. <https://playground.tensorflow.org>

II. Çoklu Sınıflı Sınıflandırma

1. Daha önce, aşağıdaki gibi iki olası seçenekten birini seçebilen ikili sınıflandırma modelleriyle karşılaşmıştık: Belirli bir e-posta spam veya spam değil, belirli bir tümörün kötü huylu veya iyi huylu olması.
2. Bu bölümde, birden fazla olasılık arasından seçim yapabilen çok sınıflı sınıflandırma modellerini inceleyeceğiz.
3. Örneğin:

- a) Bu köpek bir beagle mı, bir basset hound mu yoksa bir bloodhound mu?



- b) Bu çiçek Sibirya İris'i mi, Hollanda İris'i mi, Mavi Bayrak İris'i mi yoksa Cüce Sakallı İris mi?
- c) Bu uçak bir Boeing 747 mi, Airbus 320 mi, Boeing 777 mi, yoksa Embraer 190 mı?
- d) Bu bir elma, ayı, şeker, köpek veya yumurta resmi mi?

II. Çoklu Sınıflı Sınıflandırma

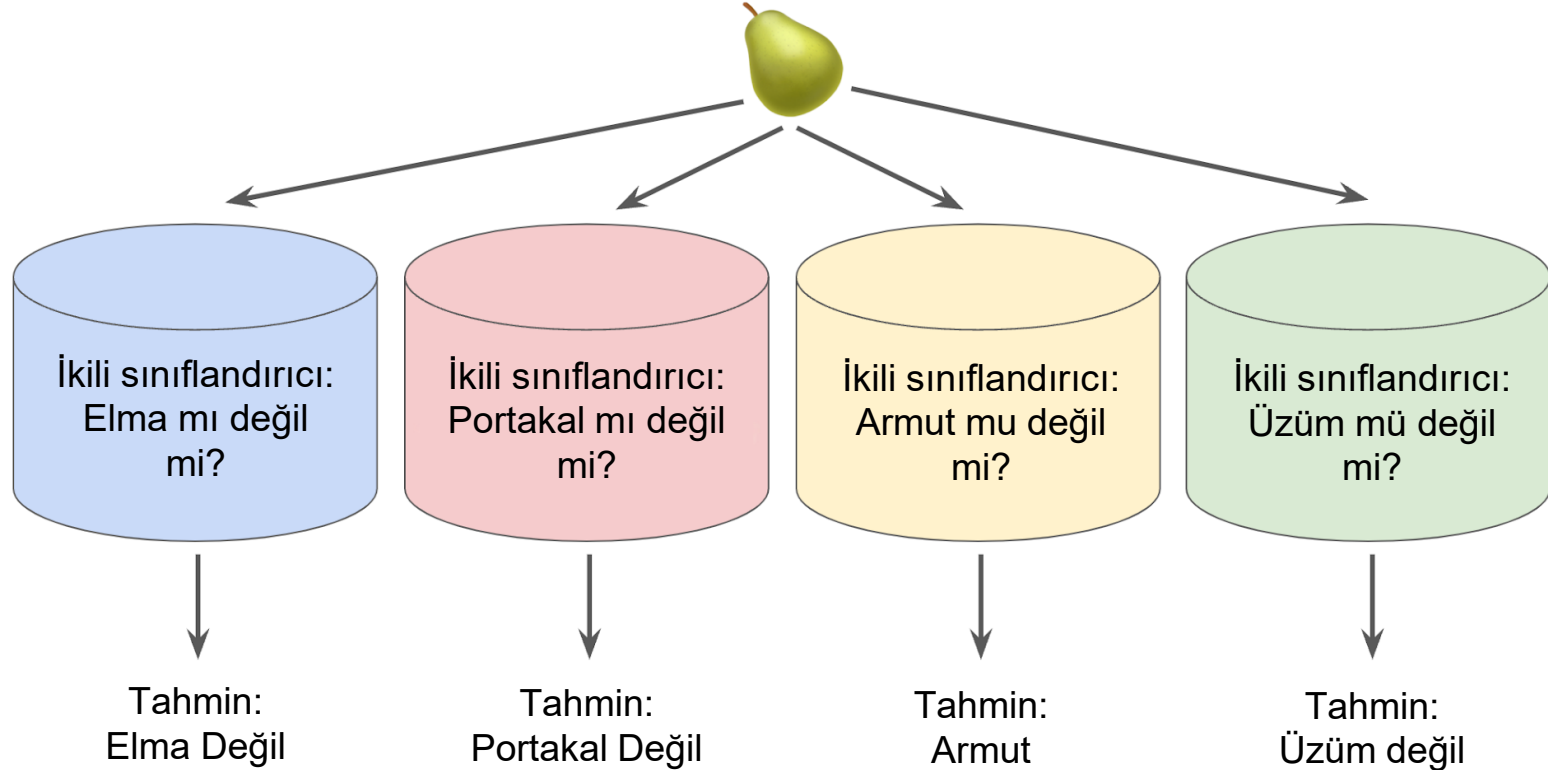
1. Bu bölümde çok sınıflı sınıflandırmanın iki ana çeşidini detaylandıracağız:
 - a) bire karşı hepsi, (one-vs.-all)
 - b) bire karşı biri (one-vs.-one genellikle softmax olarak bilinir)

II. Çoklu Sınıflı Sınıflandırma : Bire karşı hepsi

1. Bire karşı hepsi: Bire karşı hepsi, birden fazla olası etikette bir dizi evet veya hayır tahmini için ikili sınıflandırmayı kullanmanın bir yolunu sağlar.
2. N olası çözüme sahip bir sınıflandırma problemi göz önüne alındığında, bire karşı hepsi çözümü, her olası sonuç için bir ikili sınıflandırıcı olmak üzere N ayrı ikili sınıflandırıcıdan oluşur.
3. Eğitim sırasında model, her birini ayrı bir sınıflandırma sorusunu yanıtlamak üzere eğiterek bir dizi ikili sınıflandırıcıdan geçer.
4. Örneğin, bir meyve resmi verildiğinde, her biri farklı bir evet/hayır sorusuna yanıt veren dört farklı tanıyıcı eğitilebilir:
5. Bu görüntü bir elma mı?
6. Bu görüntü bir portakal mı?
7. Bu görüntü bir muz mu?
8. Bu görüntü bir üzüm mü?

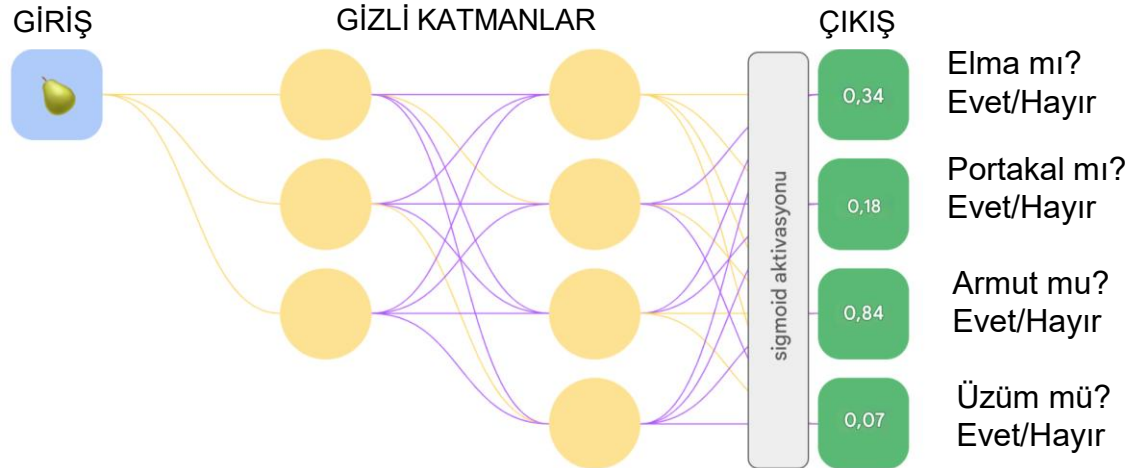
II. Çoklu Sınıflı Sınıflandırma : Bire karşı hepsi

GİRİŞ GÖRÜNTÜSÜ



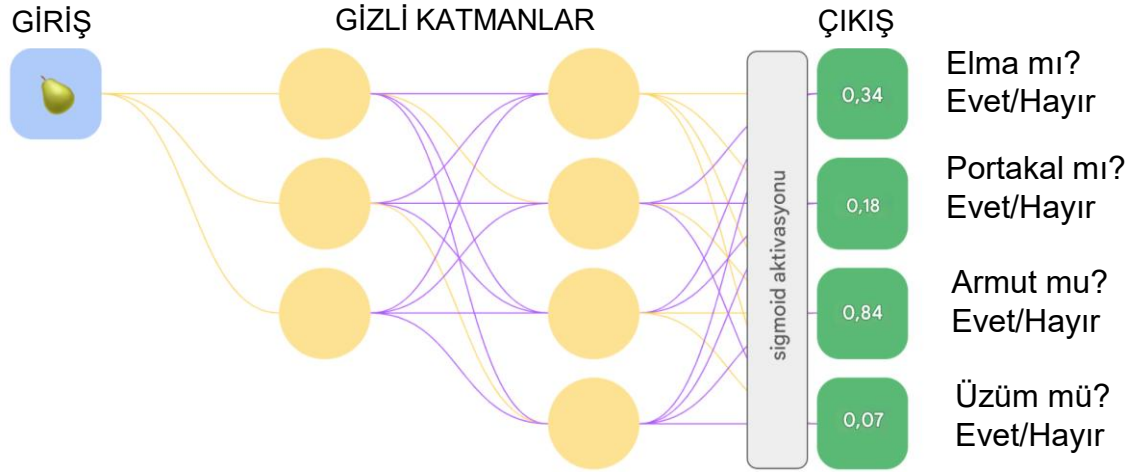
II. Çoklu Sınıflı Sınıflandırma : Bire karşı hepsi

1. Bu yaklaşım, toplam sınıf sayısı az olduğunda oldukça makuldür, ancak sınıf sayısı arttıkça giderek daha verimsiz hale gelir.
2. Her bir çıkış düğümünün farklı bir sınıfı temsil ettiği derin bir sinir ağıyla çok daha verimli bir bire-karşı-hepsi modeli oluşturabiliriz.
3. Çıkış katmanına bir sigmoid aktivasyon fonksiyonu uygulanır ve her çıkış değeri, giriş görüntüsünün belirli bir meyve olma olasılığını temsil eder.
4. Bu model, görüntünün armut olma ihtimalinin %84, üzüm olma ihtimalinin ise %7 olduğunu öngörmektedir



II. Çoklu Sınıflı Sınıflandırma : Bire karşı bir

1. Aşağıda verilen bire karşı hepsi yaklaşımda kullanılan ikili sınıflandırma örneğinde çıktı katmanındaki olasılık değerlerinin toplamının 1.0 (veya %100) olmadığını fark edebiliriz. (Aslında, toplamaları 1,43'tür.)



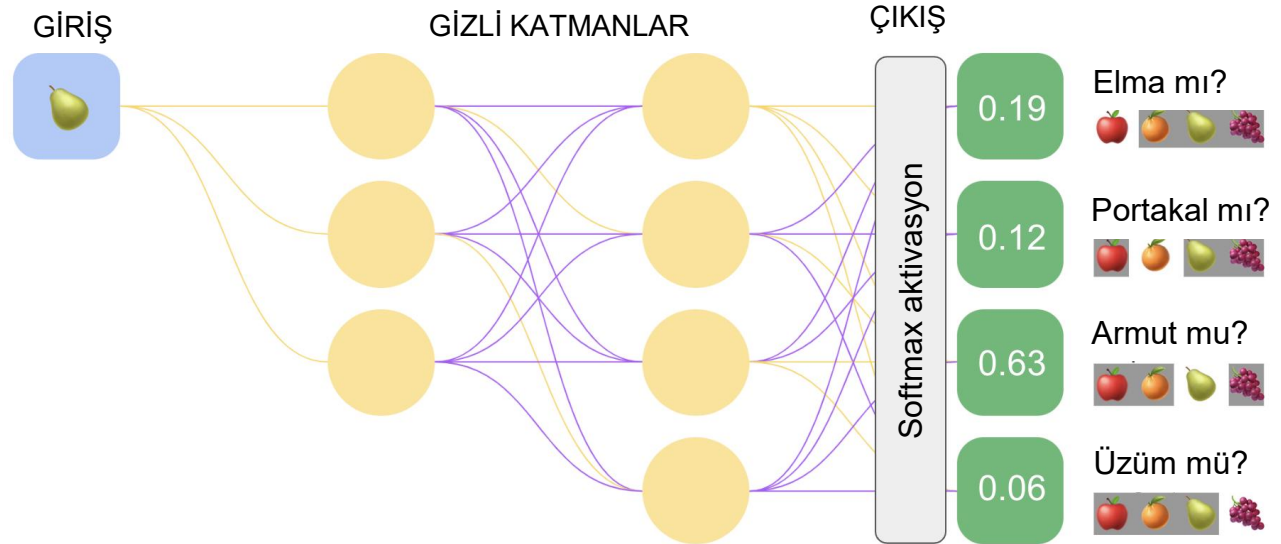
II. Çoklu Sınıflı Sınıflandırma : Bire karşı bir

1. Bire karşı hepsi yaklaşımında, her bir ikili sonuç kümesinin olasılığı diğer tüm kümelerden bağımsız olarak belirlenir.
2. Yani, “portakal”, ‘armut’ veya “üzüm” değil mi gibi diğer meyve seçeneklerimizin olasılığını dikkate almadan “elma” ile “elma değil” olasılığını belirlenir.
3. Peki ya her bir meyvenin birbirine göre olasılıklarını tahmin etmek istiyorsak?
4. Bu durumda, “elma ”ya karşı “elma değil ”i tahmin etmek yerine, “elma ”ya karşı “portakal ”ı, “armut ”a karşı “üzüm ”ü tahmin etmek isteriz.
5. Bu tür çok sınıflı sınıflandırmaya bire karşı bir sınıflandırma denir.
6. Bire karşı bir sınıflandırmayı, bire karşı tüm sınıflandırma için kullanılan aynı tür sinir ağı mimarisini kullanarak, önemli bir değişiklik ile uygulanabilir.

II. Çoklu Sınıflı Sınıflandırma : Bire karşı bir

1. Aşağıdaki şekil, bire karşı hepsi çok sınıflı sınıflandırma görevimizi bire karşı bir görev olarak yeniden uygular.
2. Softmax gerçekleştirmek için, sinir ağı yapımızda çıktı katmanından hemen önce gelen gizli katmanın (softmax katmanı olarak adlandırılır) çıktı katmanı ile aynı sayıda düğüme sahip olması gerektiğini bilmemiz gerekir.

Her çıktı değeri, girdi görüntüsünün belirtilen meyve olma olasılığını ve diğer üç meyveden herhangi biri olmama olasılığını temsil eder (tüm olasılıklar 1,0'a toplanır). Bu model, görüntünün armut olma olasılığının %63 olduğunu öngörür.



II. Çoklu Sınıflı Sınıflandırma : Bire karşı bir

1. Softmax iki seçenek olarak uygulanabilir.
 - a) Tam softmax (Full softmax) seçeneği, örneğini verdiğimiz softmax'tır; yani tam softmax olası her sınıf için bir olasılık hesaplar.
 - b) Aday örnekleme (Candidate sampling) ise softmax'in tüm pozitif etiketler için bir olasılık hesapladığı, ancak yalnızca negatif etiketlerin rastgele bir örneği için bir olasılık hesapladığı anlamına gelir.
2. Örneğin, bir girdi görüntüsünün bir elmalı mı yoksa bir armut mu olduğunu belirlemekle ilgileniyorsak, meyve olmayan her örnek için olasılık sağlamak zorunda kalmayız.
3. Tam softmax, sınıf sayısı az olduğunda oldukça uygundur ancak sınıf sayısı arttığında çok maliyetli hale gelir.
4. Aday örnekleme, çok sayıda sınıfa sahip problemlerde verimliliği artırabilir.

II. Çoklu Sınıflı Sınıflandırma : Bire karşı birçok etiket

- 1. Softmax, her öreğin tam olarak bir sınıfın üyesi olduğunu varsayar. Ancak bazı örnekler aynı anda birden fazla sınıfın üyesini temsil edebilir.**
- 2. Buna çoklu etiket denir. Bu tür örnekler için softmax kullanamayız ancak çoklu lojistik regresyonlara da güvenebiliriz.**
- 3. Örneğin, bire karşı bir modeli, her bir girdi görüntüsünün tam olarak bir meyve türünü göstereceğini varsayar: bir elma, bir portakal, bir armut veya bir üzüm.**
- 4. Ancak, bir girdi görüntüsü birden fazla meyve türü içeriyorsa (hem elma hem de portakaldan oluşan bir kase), bunun yerine birden fazla lojistik regresyon kullanmamız gerekir.**

II. Çoklu Sınıflı Sınıflandırma : Bire karşı birçok etiket

1. Başka bir örnek için bir haber makalesi örneği hem "spor" hem de "sağlık" kategorisine ait olabilir.
2. Klasik lojistik regresyonda, bir örnek tek bir sınıfa ait olabilir (multi-class).
3. Ancak çok etiketli sınıflandırmada, her etiket için ayrı bir lojistik regresyon uygulanır.
4. Her sınıf için bağımsız bir tahmin yapılır: $etiket \in \{0, 1\}$
5. Elimizde 3 etiket olduğunu varsayalım: [spor, sağlık, ekonomi]
6. Eğitilecek modeller:
 - a) Spor Modeli veya Model 1: Spor etiketini tahmin eder (etiket 1 ise spor var, 0 ise yok)
 - b) Sağlık Modeli veya Model 2: Sağlık etiketini tahmin eder
 - c) Ekonomi Modeli veya Model 3: Ekonomi etiketini tahmin eder

II. Çoklu Sınıflı Sınıflandırma : Bire karşı birçok etiket

1. Tahmin için yeni bir veri noktasına karşılık her model 0 ile 1 arasında bir olasılık üretir.
2. Belirli bir eşik değeri (örneğin 0.5) ile karar verilir: Olasılık > 0.5 ise sınıfa ait (1), olasılık ≤ 0.5 ise değil (0).
3. Örnek bir giriş vektörü şu şekilde olsun; $x = \text{"Spor salonları pandemi sonrası nasıl değişti?"}$
4. Örnek model tahminleri şu şekilde olur;
 - a) Spor modeli sonucu = 0.91
 - b) Sağlık modeli tahmini = 0.87
 - c) Ekonomi modeli tahmini = 0.21
5. Sonuç: [Spor=1, Sağlık=1, Ekonomi=0]

II. Çoklu Sınıflı Sınıflandırma : Bire karşı birçok etiket

1. Sonuç olarak çok etiketli sınıflandırma problemleri için çoklu lojistik regresyon şu şekilde kullanılır:
 - a) Her etiket için bağımsız bir lojistik regresyon modeli kurarız.
 - b) One-vs-Rest (bire karşı hepsi) gibi davranır, ama her sınıfın var-yok bilgisi ayrı öğrenilir.
 - c) Sigmoid aktivasyonu ile çoklu doğruluk tahmini yapılabilir.

Kaynaklar

- 1) Yapay Öğrenme, Ethem ALPAYDIN
- 2) Understanding Machine Learning: From Theory to Algorithms, Shai Shalev-Shwartz, Shai Ben-David
- 3) <https://playground.tensorflow.org>